Topic: Scikit-Learn

Step 0: Install Python Version

https://realpython.com/installing-python/

Step 1: Install Scikit-Learn

First, we need to install our dependency, the Python library **scikit-learn(sklearn)** which is a free software machine learning library for the Python Programming Language. The **random** library is part of the python **standard libraries** so we don't need to install it, it's already available for us!

- This is the command to install scikit-learn.

  "pip install scikit-learn"

- If there is any issue while installing we can use this command as well.

  "python -m pip install scikit-learn"

Step2: Create Your Training dataset

Usually if you have a data set, you would want to split it into a training set, validation set, and testing set. Since this is a simple example we are creating our own training set to train our linear regression model with this. The training set will contain the input and expected output values of the data set that I want to train the model on. The input will be random integer values created by the library **Random** we will use the method **randint(a,b)**, which returns a random integer N such that **a <= N <= b**. The output will be determined by a function that we will create**.**

Recall a function usually called **"f"** takes in some input/parameter value usually called "**X"** and produces some output value usually represented by **"X"**, such that "**f(y) = X".**

The function that I will create will take in multiple parameters so not just 'x' but 'a', 'b', 'c' and 'd' instead such that we get the following function.

f(a, b, c, d) = 7a + 3b + 4c + 9d = X

Example of how this function works, if we had values a=1, b=2, c=3, d=4 then X would equal 61 like the following:

f(1, 2, 3, 4) = 7(1) + 3(2) + 4(3) + 9(4) = 61

Example Of Training Set:

Input				Output
a	b	c	d	x
1	2	3	4	61
10	5	2	3	120
5	4	8	6	133
4	6	9	5	141
1	2	20	7	156
0	1	3	8	87

- This is an Example of Training a Set.

Once we have created the training set, we will split each row into two lists (a input training set , and it's corresponding output training set). So now we have two lists of all the inputs and their corresponding outputs.

Code To Generate Training Set:

```
from random import randint

TRAIN_SET_LIMIT = 1000
TRAIN_SET_COUNT = 100

TRAIN_INPUT = list()
TRAIN_OUTPUT= list()

for i in range(TRAIN_SET_COUNT):
    a = randint(0, TRAIN_SET_LIMIT)
    b = randint(0, TRAIN_SET_LIMIT)
    c = randint(0, TRAIN_SET_LIMIT)
    d = randint(0, TRAIN_SET_LIMIT)

    op = (7*a) + (3*b) + (4*c) + (9*d)
    TRAIN_INPUT.append([a,b,c,d])
    TRAIN_OUTPUT.append(op)
```

Step 3: Train Our Linear Regression Model

Next we will use the method **Linear Regression()** from the Python library **scikit-learn** to train and create our **model**. This model will try to "**model**" the function that we created for the training dataset.

Note: When we '**fit**' a function that is just another word for training.

```
from sklearn.linear_model import LinearRegression

predictor = LinearRegression(n_jobs = -1)
predictor.fit(X = TRAIN_INPUT, y = TRAIN_OUTPUT)
```

Step 4: Test Our Linear Regression Model

Now let's see if our Linear Regression function can approximate the function that we created and give us an accurate prediction (the correct answer).

X = [[10, 20, 30,40]]

The outcome should be **7*10 + 3*20 + 4*30 + 9*40 = 610**. Let's see what we got…

```
X_TEST = [[10,20,30,40]]
outcome = predictor.predict(X = X_TEST)
coefficients = predictor.coef_

print('Outcome: {} \n Coefficients: {}'.format(outcome, coefficients))
```

Here, is the Output of Outcome and coefficients is:

```
Outcome: [610.]
 Coefficients: [7. 3. 4. 9.]
```

Did you notice what just happened? The model accessed the training data, which it used to calculate the weights (coefficients) to assign to the inputs to arrive at the correct output. When giving the model test data, it successfully managed to get the correct answer!

- Here, is the complete code for the following expression of:

f(a, b, c, d) = 7a + 3b + 4c + 9d = X

```
#importing required libraries
from random import randint
from sklearn.linear_model import LinearRegression

"""Create a range limit for random numbers in the training set,
and a count of the number of rows in the training set"""
TRAIN_SET_LIMIT = 1000
TRAIN_SET_COUNT = 100

"""Create an empty list of the input training set 'X' and create
an empty list of the output for each training set 'X'"""
TRAIN_INPUT = list()
TRAIN_OUTPUT= list()

#Create and append a randomly generated data set to the input and output
for i in range(TRAIN_SET_COUNT):
    a = randint(0, TRAIN_SET_LIMIT)
    b = randint(0, TRAIN_SET_LIMIT)
    c = randint(0, TRAIN_SET_LIMIT)
    d = randint(0, TRAIN_SET_LIMIT)

#Create a linear function for the output dataset 'X'
    op = (7*a) + (3*b) + (4*c) + (9*d)
    TRAIN_INPUT.append([a,b,c,d])
    TRAIN_OUTPUT.append(op)
"""Create a linear regression object NOTE n_jobs = the number of jobs
to use for computation, -1 means use all processors"""
predictor = LinearRegression(n_jobs = -1)

#fit the linear model (approximate a target function)
predictor.fit(X = TRAIN_INPUT, y = TRAIN_OUTPUT)

#Create our testing data set, the ouput should be 7*10+3*20+4*30+9*40=610.
X_TEST = [[10,20,30,40]]

# Predict the ouput of the test data using the linear model
outcome = predictor.predict(X = X_TEST)

#The estimated coefficients for the linear regression problem. be[7. 3. 4. 9.]
coefficients = predictor.coef_
print('Outcome: {} \n Coefficients: {}'.format(outcome, coefficients))
```